

INTERNAL LEARNING AID

Product Management Fundamentals

Product management for Australian Government service delivery

SOURCE	Product Management Fundamentals — 12AUG2026, DEWR Digital Experience and Solutions
SOURCE WRITTEN BY	Simon Morris
READING TIME	about 2 hours
CONTENT REVIEWED	15 August 2026

Not an official Australian Government publication. Where this and any official departmental or Australian Government source disagree, the official source governs.

[Skip to learning content](#)

Contents

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.
- 11.
- 12.
- 13.
- 14.
- 15.

[Skip to learning content](#)

How to use this guide

Read a stage, then close the guide and try to say the core idea out loud. That small act of retrieval is worth more than reading the stage twice — it is the single most robust finding in the learning literature and the reason the interactive version withholds answers until you have committed to one.

Each stage ends with a set of contrasts: what good looks like, what usually happens, and **the tell** — an observable check you can run on your own team this week. Those are the parts worth arguing with. If a tell describes you, that is the useful outcome, not a failing.

[Skip to learning content](#)

Stage 1

Product thinking and strategy

Move from delivering scope to creating value

· 15 MINUTES

A product is an enduring vehicle for value. Product management keeps user needs, organisational outcomes and technical reality in productive tension — and it works partially before it works perfectly.

By the end you should be able to: Explain what makes something a product, distinguish product from project thinking, connect vision to strategic choices, and apply product thinking inside a project-funded environment.

Product is an ownership model, not merely software

A product can be a digital service, a policy-enabled capability, an operational system or a combination of these. What matters is that it serves identifiable users, has ongoing ownership, evolves through evidence and is judged by the value it creates. The deck puts it directly: a service is the series of interactions that helps someone do something; a product is the tool created to deliver that service.

- A project is a temporary structure for coordinating investment and delivering change.
- A product persists after launch and must be operated, measured and improved.
- A release is an output; a changed user or organisational result is an outcome.
- A product solves a problem, delivers value, supports outcomes, evolves over time and has ongoing ownership.

Worked example. Launching an application-status page is an output. Providers resolving their status questions without calling support is an outcome.

The product manager's central judgement

Product management is the intersection of business, technology and user experience — a framing that originates with Martin

[Skip to learning content](#)

Eriksson and appears on slides 8 and 54 of the deck without attribution. The role is not to collect every request or write every requirement. It is to make evidence-informed choices where user value, organisational value and technical constraints meet, and to keep the team aligned on why the work matters.

- Desirable or valuable: people have a meaningful need and will use the result.
- Viable: policy, operational, financial, legal and service constraints can sustain it.
- Feasible and usable: the team can build and operate it, and people can understand it.
- The deck names three accountabilities: vision and strategy, execution and delivery, and product success.

Vision gives direction; strategy makes choices

Vision describes the future state worth moving toward. Strategy identifies the critical problems and choices that create progress toward it. A roadmap then communicates a current sequence of bets — it is not the strategy itself. The deck is blunt about this: strategy is not doing everything, building every feature or satisfying every stakeholder. It is choosing priorities, focusing investment, making trade-offs and deciding what not to do.

- Vision should be stable enough to orient multiple delivery cycles.
- Strategy says where to focus and, crucially, what not to pursue now.
- Priorities should be traceable to evidence, constraints and desired outcomes.
- A list that accommodates every stakeholder is a decision deferred, not a strategy.

Worked example. Vision: providers can understand and manage applications with confidence. Strategy: first reduce uncertainty at the three points that generate most avoidable support demand. Deliberate non-priority: no changes to the assessment workflow this year.

Product thinking survives inside project funding

Government funds projects, not products. Departments run project frameworks, not product frameworks. Delivery is measured by outputs more often than by value. All of this is true, and none of it makes product thinking impossible — it makes partial adoption the realistic goal. The deck's own framing is that product management is not all or nothing.

[Skip to learning content](#)

- [Skip to learning content](#) can do none.
- Using outcomes imperfectly is better than tracking only outputs.

- Testing one assumption is better than testing none.
- Getting some user feedback after a release is better than none.

Worked example. A project with fixed scope and a fixed date can still run a two-week bounded discovery on the riskiest assumption, and still define one outcome measure and one guardrail before build starts.

The shift in thinking, in four pairs

The deck compresses the whole first section into four contrasts. They are worth memorising because they diagnose most disagreements you will have about priority.

— A SHIFT IN THINKING		
From	To	What changes in practice
Projects	Products	Ownership continues after go-live; someone is accountable for performance.
Outputs	Outcomes	Success is a measured change, not a delivered artefact.
Requirements	Problems	The team is briefed on the problem and constraints, not handed a solution.
Delivery	Learning	Each release is expected to produce evidence that changes the next decision.

Worked reasoning: the request that arrives as a solution

A branch head emails: 'We need a dashboard so providers can see their application status.' The weak move is to accept it, because it is specific, sponsored and easy to plan. The work then proceeds without anyone establishing what problem the dashboard solves, so success becomes 'the dashboard shipped' — which it will, on time, and the phones will keep ringing.

Product thinking does not mean refusing the request. It means holding the request and the problem apart long enough to see whether they match. Ask what the branch head has observed that makes a dashboard feel necessary. The answer here was 'providers keep calling and my staff spend half their day on it.' That is a different statement: it names a cost, a group and a behaviour. It admits several solutions — a dashboard, a group of staff, a behaviour of calling. It admits several solutions, clearer timeframes at which to deliver, and a group of staff who generate the anxiety. It also

[Skip to learning content](#)

implies a measure the dashboard alone never would: avoidable status calls.

Notice what has not happened. Nobody has been told they are wrong, no governance has been bypassed, and the dashboard may still be the answer. What changed is that success is now defined as fewer avoidable calls rather than a delivered artefact, so the team can tell afterwards whether it worked — and can stop if it does not.

- A solution arriving as a request is not a problem to fight; it is evidence that somebody feels a cost worth naming.
- The question that reframes almost anything: 'What have you seen that makes this feel necessary?'
- If the answer names a behaviour, a group and a cost, you have a problem statement. If it names a feature, keep going.

Worked example. The dashboard was built — and so were submission-time expectations and a status notification. Avoidable calls fell 41%. The dashboard accounted for less of that than anyone predicted, which is only knowable because the measure was the calls and not the dashboard.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
The team can name the outcome the work is meant to change, and who owns it after go-live.	The team can name the scope and the date, and ownership after go-live is unassigned.	Ask three people what changes for users if this succeeds. If you get three different answers, there is no outcome — only scope.
Strategy states what is deliberately not being pursued this year.	Strategy lists priorities, all of which are 'high'.	Count the things labelled top priority. More than three and the prioritisation has not happened yet.

[Skip to learning content](#)

Discovery and problem framing

Build evidence before committing to a solution

· 15 MINUTES

Discovery reduces uncertainty about users, problems and opportunities. It produces insight, not requirements — and it should change a decision, not merely generate artefacts.

By the end you should be able to: Plan discovery, distinguish four kinds of user need, tell evidence from assumption, diagnose root causes, and write testable problem, opportunity and hypothesis statements.

Understand behaviour in context

Interviews, observation, workshops, analytics, support data and surveys reveal different parts of the service. Triangulation matters because stated preferences, actual behaviour and system constraints frequently disagree.

- Research the whole journey, including non-digital channels and workarounds.
- Separate direct evidence from interpretation and inherited belief.
- Include people with varied access, capability and support needs.

Four kinds of need — and the one government creates

The deck distinguishes what users are experiencing now from what they say, what they assume and what the system forces on them. The fourth category is the one most often mistaken for a user need, and it is the one product managers in government are uniquely placed to challenge.

[Skip to learning content](#)

— WHAT ARE WE TRYING TO UNDERSTAND?

Kind of need	What it is	How you find it
Current experience	What users do now, and the tools they already use to reach their goal.	Observation, analytics, support data, walking the process.
Stated needs	What users explicitly tell you they need — for example, a mobile-responsive service.	Interviews, surveys, feedback channels.
Unstated needs	What users expect without ever asking — for example, information they can understand.	Observation of failure and hesitation; usability testing.
Created needs	What users are forced to do because of policy or the way government works.	Journey mapping and blueprinting; asking why a step exists at all.

Worked example. 'Providers need a form to request a status update' is a created need. The underlying need is to know the status. The form exists because the department has no self-service channel — that is a design decision, not a user requirement.

Artefacts are thinking tools

Personas summarise meaningful behavioural patterns; journey maps show the experience over time; service blueprints connect visible interactions to backstage people, processes and systems. None of them is the outcome. Their value is the decision they improve.

- A persona must be grounded in research, not a demographic stereotype.
- A blueprint exposes why the front-stage experience behaves as it does.
- If an artefact would not change what the team does next, it is decoration.

[Skip to learning content](#)

— WHAT EACH ARTEFACT IS FOR

Artefact	Purpose	Typically contains
Persona	Shared understanding of who the users are.	Goals, needs, behaviours, motivations, challenges, context.
Journey map	Find where improvement will have the greatest impact.	User goals, activities, touchpoints, emotions, pain points, opportunities.
Service blueprint	Understand how the service operates behind the scenes.	User actions, front-stage activities, back-stage activities, supporting systems and processes.

Discovery produces insight, not requirements

The deck draws a hard line here. Discovery produces interview notes, survey responses, analytics, feedback and observations. It does not produce requirements, features or solutions. The moment a discovery report contains a solution, it has stopped being evidence and started being advocacy.

- Discovery outputs: insights, personas, journey maps, blueprints, problem statements, opportunities, hypotheses.
- These become the inputs to solution exploration — the next stage, not this one.
- A five-whys chain moves from the visible symptom to the structural cause.

Worked example. Five whys, from the deck: providers call support → they cannot determine application status → status is not available through self-service → progress is stored across multiple systems → the service grew through several projects and was never consolidated → past investment prioritised policy commitments and functionality over end-to-end user experience.

Move from symptom to testable opportunity

A good problem statement names the affected group, their goal, the observed difficulty, where and when it occurs, and the consequence — without embedding a preferred solution. 'How might we' then reframes the problem as an invitation to explore several responses, and a hypothesis commits to one of them in a form that evidence can contradict.

- [Skip to learning content](#) Description of the current gap. Who / what / goal / where / when / why it matters.

- How might we: an invitation to explore, not a proposal.
- Hypothesis: a proposed intervention, audience, expected outcome and observable signal.
- A hypothesis you cannot imagine being wrong is not a hypothesis.

Worked example. We believe giving providers timely application-status visibility will reduce avoidable status calls. We will look for lower call volume and higher successful self-service, while checking that unresolved cases still reach support.

The Lean Canvas as an assumption model

The deck adapts Ash Maurya's Lean Canvas as a one-page way to hold the whole problem-to-measure chain while it is still uncertain. Its real function is not documentation — it is to make the riskiest assumption visible so you know what to test first.

— THE DECK'S CANVAS FIELDS	
Field	The question it answers
Problem	What problem does the organisation have that you are trying to solve?
Users	What types of users are impacted?
Solution ideas	What options could solve the problem and meet user needs?
Business outcomes	How will you know if you have solved it? What will you measure?
Hypotheses	Combine the assumptions above into testable hypothesis statements.
Riskiest assumption	For each hypothesis: what, if wrong, makes the whole thing fail?

Worked reasoning: when the analytics and the interviews disagree

Analytics show 38% of providers abandon at the identity step. Interviews say the identity step is 'fine — a bit annoying but fine.' The weak move is to pick the evidence you prefer: quantitative teams discount the interviews as a small sample, qualitative teams discount the analytics as lacking context. Both are reading a disagreement as a contest, when it is usually a signal that the two are measuring different things.

We tried to figure out what the problem was with solving it. The analytics measure even though the interviews measured eight providers recruited through an existing contact list — people who had already succeeded at identity, repeatedly, because that is how they

[Skip to learning content](#)

became reachable contacts. The sample was not too small; it was selected on the outcome being studied. That reframes the finding: the step is fine for people who can complete it, and the 38% are largely absent from the research.

The next move is not a redesign. It is to find people the drop-out represents — providers using shared devices, providers whose documents do not match the register, providers without a mobile number attached to their record. Observation of those users found three distinct failure modes, only one of which any interface change could address; the other two were data problems in a different system.

- When two sources disagree, ask what each one is a sample of before asking which is right.
- Recruiting through existing contacts selects for people who already succeed — the most common quiet bias in government research.
- 'The sample is too small' is usually the wrong critique. 'The sample cannot contain the people we are asking about' is the right one.

Worked example. The redesign the team almost shipped would have addressed roughly a third of the drop-out. Naming the other two causes moved the conversation to the register data owner, who had the actual fix and no idea the problem existed.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
The problem statement describes observed behaviour and cites where the evidence came from.	The problem statement describes a missing feature.	Remove every proposed solution from the statement. If nothing meaningful is left, it was a solution wearing a problem's clothes.
Analytics locate the problem; qualitative work explains it; the two are checked against each other.	Analytics locate the problem and the team infers the cause from the location.	Ask what would have to be true for the obvious explanation to be wrong. If nobody has checked, you have a location, not a diagnosis.

[Skip to learning content](#)

Stage 3

Outcomes, measures and OKRs

Define success before delivery begins

· 10 MINUTES

An output says what was produced. An outcome says what changed. Measurement closes the loop between intent, delivery and evidence — and if the work does not move a key result, it needs a different justification.

By the end you should be able to: Write useful outcomes and OKRs, grade them honestly, distinguish leading from lagging signals, and design measures that support learning rather than vanity reporting.

Start with the change, not the artefact

Outcomes may describe user behaviour or experience, operational performance, financial impact, risk reduction or learning. They must be specific enough to influence choices but not so narrow that teams optimise a proxy at the expense of the service.

- The deck names five kinds of good outcome: user value, operational efficiency, financial impact, risk reduction, and learning or reduced uncertainty.
- Learning counts as an outcome. Reducing uncertainty is a legitimate result of a cycle.
- Guardrail: accessibility, support demand or error risk does not worsen for other groups.

— OUTPUTS AND THE OUTCOMES THEY ARE MEANT TO CREATE

Output (the what)	Outcome (the why)
New login feature	Users can access the service faster
New form	Submission time reduced by 20%
Skip to learning content	Increased user satisfaction
Status notifications sent	Providers resolve status questions without assistance

OKRs connect direction to evidence

An objective describes a meaningful direction. Key results describe measurable evidence that the objective is being achieved. Actions are hypotheses about how to move those results; they are not substitutes for the result. The deck's formula is compact: we will [objective], as measured by [key results], via [actions].

- Objective: qualitative, specific and motivating.
- Key result: a measurable change with baseline, target and timeframe. Two to five per objective.
- Action: work the team believes will influence a key result.
- Insight: what the movement in the key result taught you.

Worked example. Objective: Make application progress understandable without assistance. KR1: Increase successful status self-service from 42% to 70% by June. KR2: Reduce avoidable status calls by 30%. Guardrail: unresolved-case escalation stays above 95%.

Grade key results honestly

The deck uses a four-point grading scale. Its value is that it makes partial achievement sayable. A team that scores 2 and explains why has produced more organisational learning than a team that quietly rewrites the target to 100%.

- Grade the key result, not the effort that went into it.
- A consistent 4 across every objective usually means the targets were set too low.
- Record the insight alongside the grade — the grade without the reason teaches nobody anything.

— OKR GRADING	
Grade	Meaning
4	Exceeded expected results
3	Sufficient achievement against expected results
2	Made progress, but fell short of expected results
1	No progress against expected results

Skip to learning content

Use a measurement system, not one magic number

Lagging indicators measure past performance and confirm an outcome after it occurs. Leading indicators are predictive measures of future performance and provide earlier signals, but they require validation. Pair them with diagnostic measures and guardrails, and define the population, denominator, baseline, period and exclusions before anyone reports on them.

- Outcome: did the desired change occur?
- Leading: are behaviours moving in the expected direction?
- Guardrail: did the intervention create unacceptable harm or push the problem somewhere else?
- Define the denominator before you publish the percentage.

— INDICATOR TYPES WITH WORKED EXAMPLES		
Type	Purpose	Example
Leading	Predictive signal of future performance	Feature usage; completion rate
Lagging	Confirmation of past performance	Customer satisfaction; outcome achieved
Guardrail	Detects harm or displacement	Escalation rate for unresolved cases; assistive-technology success rate

Worked reasoning: the measure that improved for the wrong reason

A team sets a key result: reduce median application completion time from 18 minutes to 12 by June. In April it reads 11 minutes. The weak move is to close it as achieved, because that is what the number says and the quarter is ending.

Before accepting a measure that moved, establish why it moved — because a measure has no way of telling you whether it improved for the reason you intended. Three explanations fit the same number. The form got easier, which is the intended one. Or the hardest applications stopped being submitted online and moved to the phone channel, so the online median improved by losing its slowest cases. Or a validation change now rejects incomplete submissions earlier, so half-finished attempts never enter the denominator. Only the first is a success; the second is displacement and the third is a measurement artefact.

Displacement: check whether phone volume rose when online volume fell. Here phone contacts rose 22% in the same window. The measure was

[Skip to learning content](#)

real, the improvement was not — the effort had moved rather than reduced, and the providers most affected were the ones least able to absorb it. That is why the guardrail exists: not to be cautious, but because a headline measure with no counterweight can only ever tell you good news.

- Ask of every improvement: what else would be true if this were real, and is it?
- Displacement and denominator changes look identical to success in the headline figure.
- A guardrail is chosen at the same time as the measure, because after the fact you will pick one that is already satisfied.

Worked example. The finding was not 'the change failed'. It was 'the change works for people who can complete it online, and we have made the service worse for those who cannot' — which is a different and far more useful thing to take to a governance forum.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
Every key result has a baseline, and at least one guardrail protects a group or channel.	Key results have targets but no baselines, and no guardrails at all.	Ask what number it is today. If nobody knows, you cannot demonstrate improvement later — only assert it.
The measure would change the decision if it moved the wrong way.	The measure is reported monthly and has never changed anything.	Name the last decision this metric altered. If there isn't one, it is a vanity metric regardless of how rigorous it looks.

[Skip to learning content](#)

Solution exploration and validation

Learn cheaply before you build deeply

· 15 MINUTES

Discovery asks whether a solution is valuable, usable, feasible and viable before the organisation makes an expensive commitment. A trade-off is not a compromise you regret — it is a choice you make on purpose.

By the end you should be able to: Generate options, apply the desirable/viable/feasible lenses and their trade-off pairs, expose assumptions, select appropriate experiments, and define an MVP as the smallest coherent test of value.

Diverge before you converge

Generate several ways to address the opportunity before comparing them. Workshops, co-design, brainstorming and AI-assisted ideation all broaden the option space, but evidence and constraints — not workshop popularity — must drive the decision.

- Generate many ideas; encourage diverse perspectives; avoid premature judgement; focus on outcomes.
- Separate idea generation from evaluation, in time and preferably in the agenda.
- Include non-build options: process change, policy change, guidance, removing a step entirely.
- Compare options against the intended outcome and the critical constraints, not against each other's polish.

Desirable, viable, feasible — the three lenses

The DVF model is a filter applied before the organisation invests time and resources. The ideal solution sits at the intersection of desirability (human), feasibility (technology) and viability (business and policy). Delivery fails when one perspective dominates.

[Skip to learning content](#)

— THE LENSES AND THEIR FAILURE MODES

Lens	The question	What dominance alone produces
Desirable (human)	Do users want it? Is the need validated?	User-led but unsustainable
Feasible (technology)	Can teams build and operate it?	Technically elegant but unused
Viable (business and policy)	Can the organisation sustain it?	Strategically valuable but undeliverable
None — expedience	What can we do fastest?	Easiest to build now, unlikely to deliver the outcome

The trade-off pairs tell you what you are risking

Any two lenses without the third produce a recognisable kind of failure. Naming the pair you are currently sitting on is the fastest way to say what evidence you still owe.

— COMMON TRADE-OFFS

Pair	What you get	What you are risking
Desirable + Feasible	Usable and buildable — users will use it, teams can deliver it	May not align to business or policy priorities
Feasible + Viable	Operable and sustainable — affordable to run, technically sound	May not solve a genuine user problem
Desirable + Viable	Valuable and needed — meaningful benefits, aligned to strategy	May not be technically achievable in the current landscape

Worked example. A proposal that policy loves and users need, but which requires real-time integration with three systems that batch overnight, is Desirable + Viable. The debt you owe is a feasibility spike — not another stakeholder briefing.

Map risks and assumptions

A pair of lenses that you do not value it, cannot use it, the other two lenses, or policy and business constraints make it untenable. Write assumptions explicitly and rank them by

[Skip to learning content](#)

importance and uncertainty. The riskiest assumption is the one that, if wrong, makes the whole thing fail.

- Value/desirability: will this solve a problem that matters?
- Usability: can people understand and successfully use it?
- Feasibility: can it be built and operated with the available technology and capability?
- Viability: can policy, service, legal, financial and organisational conditions sustain it?

Match the experiment to the uncertainty

An experiment is a deliberate way to obtain decision-relevant evidence. The method should follow from the kind of risk, not from what the team is comfortable running.

- Define the decision first, then the evidence that would change it.
- Test the riskiest assumption with the cheapest credible evidence.
- Predefine what result would support, weaken or kill the idea — before you run it.

CHOOSING A VALIDATION METHOD		
Risk	Method	What it tells you
Problem and value	Interviews, observation, demand tests	Whether the problem matters enough to act on
Usability and comprehension	Wireframes, prototypes, usability testing	Whether people can understand and complete the task
Feasibility	Technical spike against representative interfaces and data	Whether it can be built and operated
Viability	Operational simulation, policy and legal review, costing	Whether the organisation can sustain it
Comparative effect at scale	A/B or staged rollout with a control	Which variant actually moves the measure

Worked example. Before integrating three legacy systems, use a realistic prototype and manually supplied status data to test whether the proposed information actually reduces uncertainty for providers

[Skip to learning content](#)

What an MVP is, and what it is not

The deck defines an MVP as the smallest thing we can build that delivers value and generates learning. MVPs are focused, intentional and designed to reduce uncertainty. They are not a poor-quality solution and not an unfinished product. In government the distinction matters more than usual: 'minimum' never licenses an inaccessible, insecure or misleading experience.

- Minimum refers to scope of the bet, not to quality of the build.
- An MVP still has to meet accessibility and security obligations.
- If it cannot produce evidence that would change your next decision, it is not an MVP — it is just a small release.

Worked reasoning: choosing which assumption to test first

A team has a validated problem and four solution ideas. There is money for one test before the funding decision. The weak move is to test the idea people like most, which produces a confirmation of enthusiasm and no new information. The slightly-less-weak move is to test whatever is cheapest to test, which optimises for effort rather than for what you would learn. Neither asks the only question that matters: which uncertainty, if resolved badly, would stop this being worth doing at all?

List the assumptions the favoured option rests on, then place each one on two axes: how uncertain it is, and how much it would cost to be wrong. Assumptions that are certain do not need testing regardless of consequence. Assumptions that are uncertain but cheap to be wrong about can be resolved by building and observing. The ones worth an experiment sit in the corner where you genuinely do not know and the answer changes the decision.

For a proactive SMS notification the assumptions were: providers want it, contact data is good enough to reach them, messages will be understood without context, and the volume will not overwhelm the call centre when people ring to ask what the message meant. Desirability was near-certain from discovery. Contact-data quality was unknown and fatal — an SMS service that reaches 60% of providers is not a service. That was the test.

- Uncertain and consequential is the only quadrant worth an experiment.
- Cheapness is a tie-breaker between candidate tests, never the reason to choose one.
- If you cannot name the result that would stop the work, you are not running a test.

[Skip to learning content](#)

reached 71% of active providers, smallest organisations. The

service still went ahead — but with an assisted-channel fallback designed in from the start rather than discovered at Beta.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
At least one option in the set is non-build — process, policy, guidance, or removing a step.	Every option is a different way to build the same thing.	If all your options need engineering, you started generating after the solution had already been chosen.
The stop condition was written before the test ran.	The result is interpreted after the fact, and always supports continuing.	Ask what result would have killed this. If the honest answer is 'none', the test was theatre.

[Skip to learning content](#)

Delivery, backlogs and roadmaps

Translate outcomes into ordered, testable increments

· 15 MINUTES

Delivery is a learning system: order work toward a goal, release usable increments on a predictable cadence, measure the effect and adapt the plan.

By the end you should be able to: Structure and refine a backlog to the level of detail DES expects, work within the sprint and Program Increment cadence, use prioritisation methods with judgement, and communicate direction without turning a roadmap into a false promise.

The backlog is an ordered model of current intent

The Scrum Guide describes the Product Backlog as an emergent, ordered list and the single source of work for the Scrum Team. The deck adds the local reality: the backlog is owned and maintained by product management, can be added to by anyone in the team, is prioritised in order of value with priority set by product management, and changes as new work is identified.

- Product Goal: the longer-term target the backlog emerges against.
- Product Backlog: what may improve the product, ordered as knowledge changes.
- Sprint Backlog: the Sprint Goal, the selected items and the developers' plan.
- Anyone may add; product management orders. Those are different rights and confusing them is a common failure.

Levels exist to preserve traceability, not to create bureaucracy

Epics, features and stories move a team from outcome to increment. Note that epics, features and Program Increments are not Scrum terms — they come from scaled frameworks and local practice, and their definitions vary between organisations. What follows is how the deck defines them for DES.

[Skip to learning content](#)

— THE DES BACKLOG HIERARCHY

Level	Horizon	Definition
Epic	Many PIs, multiple teams	A measurable business outcome. Maps directly to benefits in the business case, high-level requirements and significant pain points in the service blueprint.
Feature	Within a PI, often multiple teams	A smaller tangible outcome contributing to its epic. Should result in a demonstrable output each PI.
Story	Within a sprint, one team	A small slice a team can complete and validate. Written as: as a [persona] I want [goal] so that [reason].
Task	Within a sprint	The work breakdown beneath a story.

Minimum detail at each level

The deck specifies what each backlog level must carry. This is the most directly operational content in the whole presentation — Area Path and Iteration Path are Azure DevOps fields, and getting them wrong is why work goes missing from a PI plan.

- Every level carries a hypothesis or an outcome. None of them is a specification handed down.
- Feature measurement is not optional — it is how you find out whether the feature worked.
- Iteration path must align to the sprint the work is actually expected in, or PI reporting silently drifts.

— MINIMUM DETAIL

Level	Required fields
Epic	Epic hypothesis; epic description; business outcomes and benefits; objective. Hypothesis-driven, expressed as the outcome delivered when the work is complete.
Feature	Feature hypothesis; sizing estimate; acceptance criteria; in/out of scope; feature measurement (e.g. '30% reduction in processing time'); risks, issues and dependencies.
Story	Story description in as-a/I-want/so-that form; story points; area path; iteration path aligned to the sprint in which delivery is expected.

[Skip to learning content](#)

The DES delivery cadence

DES maintains a common planning and delivery cycle so teams synchronise. Two-week sprints deliver small increments of value; ten-week Program Increments deliver key business outcomes. Knowing this cadence is what makes the backlog levels meaningful — a feature is 'PI-sized' only because a PI is ten weeks.

- The PI flow: business outcome → candidate features from the backlog → refinement → increment planning → delivery → measurement.
- Refinement confirms the detail, scope and feasibility of each feature before the increment starts.
- Five sprints fit inside a PI, which is why feature sizing and iteration paths have to be honest.

— CADENCE		
Cycle	Length	Delivers
Sprint	2 weeks (10 working days)	Small increments of releasable value
Program Increment (PI)	10 weeks (50 working days)	Key business outcomes

Discovery runs an increment ahead of delivery

Continuous delivery relies on parallel streams. Product management works on discovery and solution exploration an increment in advance, while delivery teams build the features identified from the previous refinement cycle. The two streams share outcomes, people and feedback — otherwise you get a discovery function producing documents nobody builds.

- Discovery for PI N+1 happens during delivery of PI N.
- Releases occur throughout, not only at PI boundaries.
- Operational data and research from live releases feed back into the opportunity and product backlogs.
- Continuous does not mean chaotic — it means the loops overlap on a known cadence.

Prioritisation is economic judgement made explicit

Impact/effort and WSJF make trade-offs discussable; they do not manufacture certainty. SAE defines WSJF as relative cost of delay divided by relative job duration or size. Scores should be relative, evidence-informed and revisited — prioritisation is not a one-off activity.

- [Skip to learning content](#) higher score sequences earlier.

- Smaller valuable work often moves earlier because it creates learning and benefit sooner.
- Impact/effort quadrants: quick wins, big-ticket items, fill-in jobs, thankless tasks.
- Dependencies, safety, legislated mandates and confidence still require judgement over the score.

— THE WSJF COMPONENTS

Component	Contributes to
User and business value	Cost of delay (numerator)
Time criticality	Cost of delay (numerator)
Risk reduction and opportunity enablement	Cost of delay (numerator)
Job size or effort	Denominator

A roadmap communicates direction at the right confidence

Outcome roadmaps emphasise objectives and key results over time horizons; feature roadmaps specify likely solution work against PIs. The deck shows both. Use Now/Next/Later or similar confidence bands where exact dates would imply certainty the evidence does not support.

- Outcome roadmap: Now / Next / Later, each carrying objectives and their key results.
- Feature roadmap: PI X / PI Y, each carrying candidate features under an objective.
- The further out, the lower the confidence and the coarser the commitment should be.
- A dated 18-month feature list is a forecast presented as a promise.

Worked reasoning: the roadmap that promised dates it could not keep

The published roadmap shows four features with quarterly dates. Discovery has just invalidated the hypothesis behind the second one. The weak move is to keep the date and quietly change what ships under that name, which preserves confidence for one quarter and destroys it permanently the first time somebody notices. The other weak move is to stop publishing a roadmap, which reads as evasion and removes the only artefact stakeholders have for planning their own work. Both mistakes come from treating a roadmap as a commitment register rather than a confidence.

[Skip to learning content](#)

Rebuild it around what you actually know. Work in the current increment has been refined, sized and started: you can commit to that, and dates are meaningful. Work in the next increment has a validated problem but unproven solutions: you can commit to the outcome and not the shape. Work beyond that is a direction. Publishing all three with the same visual weight and the same date format is the lie — not the dates themselves.

The invalidated feature moves out of the committed band and its outcome stays, with an honest note that the approach did not survive testing and the team is exploring others. The conversation with the sponsor then becomes about the outcome and the evidence rather than about a slipped date, which is a conversation you can actually have.

- A roadmap is a confidence statement. Presenting three confidence levels in one format is what makes it a false promise.
- 'What would have to be true before a date is meaningful?' converts a date demand into a useful conversation.
- Removing an approach while keeping its outcome shows the work is being steered, not abandoned.

Worked example. The sponsor's actual concern turned out to be a Ministerial commitment tied to the outcome, not the feature. Once that was on the table the team could propose a different route to the same outcome — which the date-defence conversation had been hiding for two months.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
Every feature carries a measurement field, and it is filled in.	Features carry acceptance criteria, so you learn whether it was built but never whether it worked.	Open the last three completed features. If none records what changed, the loop is not closing.
The roadmap uses confidence bands beyond the current increment.	The roadmap gives dates for everything, and the far ones are quietly known to be fiction.	Ask what evidence supports the date twelve months out. If it is 'that is when the funding ends', it is a constraint, not a forecast.

[Skip to learning content](#)

Lifecycle and continuous learning

Connect discovery, delivery, release and improvement

· 10 MINUTES

The lifecycle is a loop: discover problems, define solutions, deliver value, measure outcomes, iterate and improve.

By the end you should be able to: Relate the product lifecycle to the DTA service design and delivery process, and explain why discovery and measurement continue after release.

Solve the right problem; build the right thing; build it right

These are linked responsibilities, not three isolated departments.

Product, design and technology need shared context so that feasibility knowledge improves discovery and user evidence improves delivery.

The deck frames the whole lifecycle as moving a service from 'need identified' to 'need met'.

Government phases manage increasing confidence

The DTA service design and delivery process moves through Discovery, Alpha and Beta/Live. Discovery understands users and constraints; Alpha explores and tests different ways to meet those needs; Beta builds and trials an accessible, secure service; Live sustains and improves it. Note the deck presents Beta and Live as a combined stage — this course separates them because the ownership question at go-live is where services most often fail.

- A phase is not permission to stop learning.
- The Digital Service Standard should be considered across the whole lifecycle, not at a single gate.
- Non-digital channels and end-to-end service impacts remain in scope throughout.

[Skip to learning content](#)

— DTA PHASES

Phase	Purpose	Typical evidence produced
Discovery	Understand users, policy intent and constraints	Insights, problem statements, opportunities
Alpha	Explore and test several approaches	Prototypes, validated or killed hypotheses
Beta	Build and trial an accessible, secure service	Live usage under real conditions, performance data
Live	Operate and continue improving	Ongoing performance, research and feedback

Continuous does not mean chaotic

Teams can explore upcoming opportunities while delivering validated work, provided both streams share outcomes, people and feedback. Reviews, operational data and research continuously update the opportunity and product backlogs. The loop is: discover problems or opportunities, define solutions, deliver value, measure outcomes, iterate and improve.

Worked reasoning: the service that went Live and quietly decayed

A service launches, meets its targets, and the project closes. Eighteen months later completion is down 8% with no release in that period and nobody can say why. The weak reading is that the original design was wrong, which is both unfalsifiable and useless. The code did not change, so something around it did — and that is the whole argument for treating Live as a phase with an owner rather than as an ending.

Work outward from what could have moved. Users change: a new cohort with different circumstances enters the service. Policy changes: an eligibility rule shifts and the guidance no longer matches the form. Upstream systems change: a register the service reads starts returning a field differently. Volume changes: a seasonal peak pushes a queue past a threshold nobody set an alarm on. Each is invisible to a team that no longer exists, and each is obvious within a week to an owner watching a small number of measures.

Here it was policy. A rule change had added a condition the form asked about in language taken from the legislation, and completion fell among exactly the people the change was meant to help. No release had
bro
wh
[Skip to learning content](#) only stopped matching the world, service nobody is holding.

- A measure that drifts without a release means the world changed, not the code.
- Users, policy, upstream systems and volume — the four things that move underneath a stable service.
- The Beta-to-Live handover is where ownership is either named or lost; everything above follows from which one happened.

Worked example. The fix was a wording change and took a day. Finding it took four months, because there was no owner, no measure being watched, and no cadence at which anyone would have looked.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
A live service has a named owner, a measure and a review cadence.	A live service has a support rota and an SLA for availability.	Ask who decides what gets improved next. If the answer is a team that no longer exists, decay has already started.

[Skip to learning content](#)

Stage 7

Roles, teams and decision rights

Put accountability close to the evidence

· 10 MINUTES

Strong product work is cross-functional. Role clarity should make collaboration and decisions easier — not create hand-offs between silos.

By the end you should be able to: Distinguish complementary product roles, name the surrounding delivery and governance roles, explain the product trio, and design decision rights across strategic, coordination and delivery layers.

Product manager and Product Owner are different horizons

Organisations define these titles differently, and arguing about the definitions is usually a proxy for an unresolved accountability question. In the deck's model the two share a common objective but focus on different horizons.

- The deck's two guiding questions: the PM asks 'are we working on the right things?'; the PO asks 'do teams have everything they need to deliver successfully?'
- Do not use titles as a substitute for explicit accountability.
- Avoid making the Product Owner a requirements courier.
- Avoid separating strategic discovery from the team that must deliver and operate the result.

[Skip to learning content](#)

— THE SPLIT	
Product Manager	Product Owner
Vision	Backlog management
Strategy	Story refinement
Outcomes	Acceptance criteria
Roadmaps	Sprint support
Stakeholder alignment	Clarifying requirements

The trio combines product, design and technology

A product trio brings viability, desirability and feasibility into the same discovery conversation — product manager (viable), designer (desirable), technology lead (feasible). It is a collaboration pattern, not a committee that replaces broader research, delivery and stakeholder expertise. The deck shows trios operating at all three organisational layers, not only in delivery teams.

Three layers, three questions

The deck separates the organisation into strategic, coordination and delivery layers, each with its own question, scope and accountability. Escalate constraints, not every routine decision.

- State who decides, who contributes evidence and who must be consulted.
- Place reversible, local decisions closest to the team.
- Reserve governance attention for material risk, investment and cross-service trade-offs.

[Skip to learning content](#)

— HOW THE LAYERS WORK TOGETHER

Layer	Its question	Responsible for	Typical work
Strategic	What future are we trying to create?	Direction, user needs and service outcomes — the why	Service strategy, cross-product experience, future service design, long-term technology direction, emerging user needs, organisational outcomes
Solution / coordination	What should we deliver next?	Translating strategy into roadmaps, priorities and features — the what and when	Roadmaps, feature prioritisation, dependency management, exploring solution options, product-level outcomes, cross-team coordination
Tactical / delivery	How do we successfully deliver this work?	Design, build, test and release — the how and how much	Refinement, sprint planning, story development, quality assurance, user validation, sprint showcases

The roles around you

Product managers in DES do not operate alone, and the deck names the surrounding structure explicitly. Knowing who owns what saves you from either escalating unnecessarily or making a decision that was never yours.

[Skip to learning content](#)

AND 92 — THE WIDER DELIVERY AND GOVERNANCE STRUCTURE

Role	Layer	Owns
Senior Responsible Officer	Governance	The project outcomes, delivery roadmap, championing the change
Senior Supplier	Governance	Supplier-side commitment and capability
Project Board	Governance	Investment and material risk decisions
Lead Delivery Manager	Coordination	Delivery across teams; removing systemic impediments
Product Manager	Coordination / strategic	Outcomes, priorities, product direction
Product Owner	Delivery	Building, curating and prioritising a refined, validated backlog
UX Lead / Designers	Solution	User research, service and interaction design
Solution Architect / Architects	Solution	Technical approach and integrity
Scrum Master	Delivery	Team process, flow and impediment removal
BAs, Developers, Testers	Delivery	Making working increments

Worked reasoning: the Product Owner who became a courier

The delivery team asks a question in refinement. The Product Owner says they will find out, goes to the business area, returns two days later with an answer. This repeats. Velocity drops, the Product Owner is working late, and the obvious diagnosis is that they need better time management or the team should ask fewer questions. Both are wrong, and both blame a person for a structural fault.

Nobody has agreed who is allowed to decide what. In the absence of that agreement, every question becomes an escalation by default, and the Product Owner becomes a relay rather than a decision-maker.

Sort the questions from a fortnight of refinement into three piles and the shape appears. Some are reversible and local — the wording of a value stream map, the definition of a field. Those should be decided in the room, by the people in it, and changed later if they are wrong. Some are business rules with a genuine owner: eligibility thresholds, retention

[Skip to learning content](#)

periods. Those need a named person and an agreed response time, not an ad-hoc hunt. A few are genuinely strategic — whether to support a channel at all — and belong at a forum. The fix is one conversation naming which pile is which, and who holds the second. The Product Owner's week changes immediately, and so does the team's, because the default stops being 'ask and wait'.

- Reversible and local decisions belong with the people closest to the work; the cost of being wrong is a later change.
- Business rules need a named owner and an agreed response time, not goodwill.
- If decision rights are unstated they default to whoever is most persistent or most available — neither of which is a design.

Worked example. Of 31 questions raised in one fortnight, 24 were reversible and local. The team had been treating all 31 the same way because nobody had ever said they could decide anything.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
Reversible decisions are made by the team; governance sees material risk and investment.	Everything is escalated, so governance is busy and slow on the things that matter.	Look at the last governance agenda. Count items that could have been decided by the team without meaningful risk.
The Product Owner spends most of the week on refinement and acceptance.	The Product Owner spends most of the week relaying questions between the team and the business.	If the PO is the only route to an answer, decision rights were never agreed — that is a structural problem, not a workload one.

[Skip to learning content](#)

Product practice in government

Work with policy, assurance and delivery constraints without losing the outcome

· 15 MINUTES

Government product practice combines user-centred evidence with public value, policy intent, stewardship, assurance and whole-of-service responsibility.

By the end you should be able to: Apply product principles inside DES ways of working and delivery phases, use the current Digital Service Standard accurately, distinguish it from internal governance, and protect inclusion, accessibility, security and service continuity.

Constraints are design inputs

Policy intent, legislation, privacy, security, accessibility, records, procurement, funding and legacy systems shape the feasible solution space. Surface them early in discovery so they inform options rather than arriving as late-stage blockers.

- A policy requirement is not automatically a user need.
- Compliance says what must be true; design still determines how people experience it.
- Whole-of-service outcomes include assisted and non-digital channels.

Ways of Working: behaviours and framework

DES ways of working has two core elements. The behaviours describe how people are expected to operate; the Digital Delivery Framework is the end-to-end, repeatable delivery pipeline that ensures consistent practice.

- We put user needs at the centre.
- We are accountable and take ownership for solving issues.
- We empower our people.
- We connect and work across boundaries.
- [Skip to learning content](#)
- We are innovative and contemporary.

The seven principles in practice

These are the division's stated operating principles. Each one has an implication that changes what a product manager actually does — which is why they are worth knowing by number, not just by sentiment.

— PRINCIPLES AND THEIR IMPLICATIONS		
#	Principle	What it means in practice
1	Start with the problem, not the solution	Understand user needs, policy intent and root causes before committing to solutions. Discovery comes before build.
2	Validate before you scale	Reduce risk through prototypes, testing and incremental releases rather than a single large commitment.
3	Deliver value early and often	Prefer incremental delivery of valuable outcomes over large, infrequent releases.
4	Decisions belong closest to the problem	People with the greatest understanding of users and delivery realities make day-to-day decisions.
5	Focus on outcomes, not outputs	Measure success by value created, not volume of work completed. Outcomes over features, benefits over deliverables, value over activity.
6	Design for the whole service	Optimise the end-to-end experience rather than individual systems, projects or organisational silos. Reuse before rebuild.
7	Learn continuously	Discovery, validation and improvement continue throughout the life of a service.

The four DES delivery phases

DES has an agreed end-to-end delivery pipeline for all projects, divided into four phases with stage gates between them. This is the pipeline your work actually sits inside, and it is distinct from both the DTA service phases and the Digital Service Standard.

- The stated purpose: projects are well planned with the right amount of detail, teams have a clear runway, there is a shared view of success, and stage gates build in quality and predictability.
- The product risk sits at Closure — a project closes, but the product does not.
- Do not confuse these four phases with the DTA's

[Skip to learning content](#)

the Digital Service Standard. They
ist.

— THE DES PIPELINE

Phase	Focus	Product management contribution
Pre-Approval	Establishing the case for investment	Problem definition, user evidence, outcome hypotheses, options and their risks
Pre-Delivery	Planning with the right amount of detail; establishing the runway	Refined backlog, validated approach, measures and baselines defined before build
Delivery	Building and releasing increments	Ordering the backlog, protecting the outcome, running discovery an increment ahead
Closure	Transitioning and completing the project	Handing over enduring ownership, measures and improvement cadence — not just documentation

Use the current Digital Service Standard accurately

Version 2.0 has 10 criteria, reduced from the former 13, and forms part of the Digital Experience Policy. It expects agile, user-centred work and consideration of the criteria across the service lifecycle. It is now fully in effect: it applied to new and redeveloped services from 1 July 2024, and to public-facing services that already existed before that date from 1 July 2025.

- The standard covers accessibility, inclusion, security, trust, adaptability and measurement.
- Internal DES delivery phases can coexist with this external standard but should not be mistaken for it.
- Governance evidence should demonstrate how the criteria are being met, not merely that a template was completed.
- Staff-facing internal services sit outside the Phase 2 scope; check current DTA guidance before assuming applicability.

Make governance a learning and risk conversation

Strong gates examine evidence, unresolved risk, outcome confidence and readiness for the next investment — not only schedule and document completion. Product evidence can strengthen business cases and assurance decisions rather than competing with them.

- Show what was learned and what changed as a result.
- Make residual assumptions and guardrails visible.
- Preserve a clear owner for benefits and service performance after

[Skip to learning content](#)

- Completeness will approve a well-documented failure.

Worked reasoning: the policy requirement that made things worse for users

Policy requires a declaration screen with specific wording before submission. Research shows providers do not understand it, skim it, and a proportion abandon at that point. The weak responses sit at two extremes. Refusing to implement it treats a legitimate policy instrument as an obstacle and is not a decision a product manager gets to make. Implementing it without comment discards evidence that the department paid to gather and would want. Neither is the job.

Separate the intent from the implementation, because a policy requirement almost always specifies both and only one of them is actually binding. The intent here is that providers make an informed declaration and that the department can demonstrate they did. The implementation is a particular screen with particular words. The evidence does not challenge the intent — it shows the implementation is failing to achieve it, since a declaration nobody understands is not informed consent, and the department's assurance position is weaker than it looks.

That reframes the conversation with the policy owner from 'we would like an exemption' to 'the current wording is not producing the thing you need, and here is what people did.' The move is to meet the intent with an evidence-informed design, keep the mandated wording where it must appear, and take the comprehension data back to the owner as a finding rather than a complaint.

- Policy usually specifies both an intent and an implementation. Establish which is binding before proposing anything.
- Evidence that an implementation fails its own intent is useful to the policy owner, not hostile to them.
- Take friction data to the owner as a finding. An exemption request invites a no; a finding invites a conversation.

Worked example. The mandated sentence stayed, in a plain-language frame explaining what it meant and why. Comprehension went from 31% to 78% in testing, and the policy owner used the result to revise the standard wording department-wide.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
Gate papers show what was learned, what changed as a result, and what is still uncertain	Gate papers show that the required documents exist and are signed.	Search the last gate paper for the word 'assumption'. Zero mentions usually means the risks were smoothed rather than surfaced.

[Skip to learning content](#)

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
Accessibility is tested during Beta and blockers are fixed before Live.	Accessibility is assessed near Live and issues become post-launch enhancements.	Ask when the first assistive-technology test is scheduled. If it is after the build is complete, it is an audit, not design.

[Skip to learning content](#)

Integration and capstone

Run the whole chain from need to measured value

· 10 MINUTES

Product management is a connected chain of decisions: need → problem → outcome → options → evidence → ordered delivery → measurement → learning.

By the end you should be able to: Integrate the course into a defensible product approach and explain the reasoning to users, delivery partners, stakeholders and governance.

Maintain the chain of evidence

Every major item should trace backward to an outcome and forward to a way of learning. If a feature cannot be connected to a user or service problem, an intended result and a measure, its priority is unclear. The deck's closing questions are the shortest version of this test: what problem are we solving, what outcome are we trying to create, what evidence do we have, and what have we learned?

What good looks like

The deck names six qualities. They are worth treating as a checklist for your own work rather than as a poster: outcome focused, user centred, evidence based, collaborative, adaptive, value driven.

- Outcome focused: success is a measured change, not a delivered artefact.
- User centred: decisions are grounded in what users actually do.
- Evidence based: claims are traceable to something observed.
- Collaborative: product, design and technology shape options together.
- Adaptive: the plan changes when the evidence does.
- Value driven: work that does not move a result needs a different justification.

Tell the product story at three levels

Ex [Skip to learning content](#) ce, investment and risk. Delivery teams need the problem context, goal, guardrails and near-term

decisions. Users and operational partners need clear changes, support and feedback paths.

- Keep one truth; adjust the depth, not the facts.
- Explain trade-offs and confidence honestly.
- Separate evidence, assumption and decision in the way you speak.

Reflection becomes practice

Mastery comes from repeatedly making and explaining product decisions. The capstone uses the provider application-status case that runs through the source deck so you can practise the complete method, self-assess against a rubric, and export a reusable product brief.

- The deck's reflection questions: what problem are you solving and is it the right one? How are you measuring success? What assumptions need validation? What feedback loops exist? How will you know you have delivered value?
- Retrieval and spacing are why this system makes you recall before it lets you reread.

Worked reasoning: reporting a result you did not want

The headline measure improved 14%. Broken down, one cohort — providers using assistive technology — got materially worse. The board meets on Thursday and asked for the headline. The weak move is to report 14% and mention the cohort in an appendix, which is technically complete and functionally a lie, because the number that gets remembered is the one in the first sentence. The move that feels brave and is actually careless is to lead with the failure, which invites cancellation of work that is mostly succeeding. Neither serves the decision the board is there to make.

Lead with what changes what they do. The board's job is to decide whether to scale this to all providers next quarter. The 14% says scaling is worth considering; the cohort result says scaling now would extend a harm to a larger group, and that is decision-relevant in a way the headline is not.

So the sentence is: the intervention works, it is not yet safe to scale, here is what we need to establish first, and here is when we will know. That is not softening the finding — it is putting it where it acts. It also protects the work, because a board that learns about the cohort later, from somebody else, will not remember that it was in an appendix.

- Lead with the thing that changes the decision in front of the audience, not the thing that is most flattering or most dramatic.
- [Skip to learning content](#) group is not a summary; it is a different

- Say what you need to establish and by when. A finding without a next step reads as either panic or excuse.

Worked example. The board approved a four-week extension rather than the scale-up. The cause was a single component that had never been tested with a screen reader, and it was fixed inside that window — which a cancelled programme would not have been.

Check yourself

WHAT GOOD LOOKS LIKE	WHAT USUALLY HAPPENS	THE TELL
Priority decisions can name the evidence that would reverse them.	Priority decisions cite a scoring model or a sponsor.	Ask what would change your mind. An answer you cannot give is a decision that cannot be argued with — which is not the same as a good one.
Results are reported disaggregated, so harm to a subgroup is visible.	Results are reported as averages, and the average is genuinely better.	Split the headline number by channel, assistive technology and caseload size. If nobody has, you do not yet know who you helped.

[Skip to learning content](#)

Worked cases

Provider application status

The deck's running case, worked end to end

Providers phone the support centre constantly to check where participant applications have got to. The obvious response is a status page. This case follows the chain properly and arrives somewhere better than the obvious response.

Stage 1 — Framing it as a product, not a ticket

The request arrived as 'build a status page', already scoped and already funded as a project deliverable. The product manager's first move was not to refuse it but to ask what would still be true in two years: who owns the status experience, who notices when it degrades, and what happens when the underlying systems change again.

Notice. The request was not wrong, it was premature. Reframing it as an owned capability rather than a deliverable is what created room to ask why the calls happen at all.

Stage 2 — Finding the actual difficulty

Call-reason coding said 'status enquiry' was the top driver. Twelve provider interviews and four hours of contact-centre observation said something more specific: providers were not confused about where an application was, they were unable to tell whether they needed to do anything. Most calls ended with the agent saying 'nothing needed at your end'.

Our providers are currently experiencing uncertainty about whether action is required from them when supporting a participant's application. This occurs across the provider portal and the support line, and has been happening since the 2023 platform split. This is critical because it generates avoidable support demand and delays participants. Evidence: call-reason coding, 12 interviews, 4 hours of contact-centre observation.

Notice. The stated problem was 'I cannot see the status'. The real problem was 'I cannot tell whether it is my turn.' A status page answers the first and not the second.

Stage 3 — Defining success before building

The team wrote the objective around resolution rather than deflection,

[Skip to learning content](#) easy to fake by making it harder to call. A guardrail was set on the assisted channel so that any improvement achieved by pushing demand elsewhere would show up immediately.

Objective: providers can tell whether action is required, without contacting us.

KR1: avoidable status contacts down 30% (baseline 4,100/month).

KR2: providers correctly state their next action unprompted in 8 of 10 usability sessions.

Leading: proportion of sessions reaching the next-action panel.

Guardrail: median time to resolution for unresolved cases does not worsen.

Notice. The guardrail is the part that gets cut under pressure and the part that later proves you did not simply displace the problem.

Stage 4 — Testing the cheap thing first

Three options went on the table: the requested status page, an action-required notification, and a non-build option of changing the provider's default view. The riskiest assumption was shared by all three — that providers would trust departmental status enough to act on it. A prototype with manually supplied data tested comprehension and trust before any integration work was scoped.

Decision: whether to fund the status integration.

Riskiest assumption: providers will act on departmental status rather than ringing to confirm it.

Method: clickable prototype, manually supplied data, 8 representative providers.

Proceed if 6 of 8 state the correct next action unprompted AND say they would not call to confirm.

Notice. The expensive integration was never the question. Whether the information would be believed was the question, and it cost eight conversations to answer.

Stage 5 — Ordering the work honestly

The feature was sized to a single PI and written with a measurement field from the start. A dependency on the assessment platform's status API was recorded as a risk rather than assumed away, and the roadmap said Now/Next/Later rather than naming dates the evidence could not support.

Feature hypothesis: providers shown a plain-language next action will resolve without contacting support.

Measurement: successful self-service rate for status queries, target 65% in the first PI after release.

In scope: application progress and required action. Out of scope: payment

[Skip to learning content](#) refresh frequency from the assessment platform.

Notice. The measurement field is what made the next stage possible. Without it the PI would have closed with the benefit assumed.

Stage 8 — Meeting the standard without theatre

Accessibility testing during Beta found that status was conveyed by colour alone in the first build. It was fixed before Live rather than logged as an enhancement. At the gate the team presented the finding, the fix and the residual risk on data freshness rather than a completed template.

Notice. Presenting a problem you have already fixed builds more governance confidence than presenting a clean template. It also makes the next honest report easier.

Stage 9 — What the evidence said afterwards

Avoidable contacts fell 34%. The guardrail held: time to resolution for unresolved cases was flat. Usability sessions reached 9 of 10 on next-action comprehension. One finding was uncomfortable — providers with the largest caseloads still called, because they were checking several participants at once and the design assumed one.

Notice. The result was good and incomplete. Naming the caseload finding turned a successful release into the start of the next cycle rather than the end of the project.

The team ended up building roughly what was originally asked for, but with a different core: an action-required signal rather than a status display. That difference came entirely from stage 2, and would have been invisible if the work had started at stage 5.

Employer vacancy drop-off

A case where the method catches the mistake — late

Employers were abandoning the vacancy-advertising form at high rates. The team fixed the form, the completion rate improved sharply, and the outcome did not move at all. This case is included because a method that only ever endorses itself teaches nothing.

Stage 2 — Where the team started — and stopped

Analytics showed 61% abandonment at the position-details step. The team ran a heuristic review, found the step long and the wording dense, and moved straight to redesign. No employer was interviewed. The reasoning felt sound, the data showed exactly where people left.

[Skip to learning content](#)

E. Nobody established WHY. The team treated a location as a diagnosis — the single most common

discovery failure.

Stage 3 — The measure that let it happen

Success was defined as form completion rate. It was specific, baselined and time-bound — it looked like a good key result and satisfied every formal test anyone would apply to it. What it never asked was whether a completed form produced a filled vacancy. Completion is a step in the employer's journey, not the thing the employer came for, and the two can move in opposite directions: a shorter form is easier to finish and can carry less of the information a jobseeker needs to decide whether to apply. Nothing in the measure would report that, which is precisely why the measure survived review.

Objective: make advertising a vacancy easier.

KR1: completion rate 39% → 65% by March.

(no guardrail, no outcome measure beyond the form itself)

Notice. A well-formed key result measuring the wrong thing is more dangerous than a vague one, because it passes review and then drives the work.

Stage 5 — A clean delivery of the wrong thing

The redesign shipped in one PI. Completion rose from 39% to 71%, comfortably beating the target. The team graded the key result a 4, closed the epic and moved on. Delivery performance was genuinely excellent and nobody involved did anything careless. That is what makes this the instructive case. Every signal the team had agreed to watch said the work was succeeding, and the signal that would have contradicted it — whether vacancies were being filled — had never been agreed as something to watch. A team cannot notice what it has not instrumented, and by the time the epic closed there was no longer anyone looking.

Notice. Every delivery signal was green. This is what makes output-based measurement so persistent: it rewards teams accurately for the wrong achievement.

Stage 9 — The finding nobody was looking for

Six months later an unrelated analysis showed vacancies filled had not moved. Digging in: the abandoned step had been acting as an unintentional filter. Employers who found it hard were largely those posting roles with pay or conditions that would not attract candidates. Making the form easier produced more listings, not more filled vacancies — and increased assessment workload downstream.

[Skip to learning content](#)

...nobody had noticed. Removing it ...led the service, which is the exact

failure a guardrail on a downstream outcome would have caught within weeks.

Stage 3 — What the correction looked like

The team rewrote the measure set around the outcome the service exists for, kept the completion improvement, and added the guardrail that should have been there. Crucially they did not revert the redesign — the form genuinely was better, it just was not sufficient.

Objective: employers fill roles through the service.

KR1: vacancies filled within 30 days 22% → 30%.

Leading: completion rate (retained – it moves early).

Guardrail: downstream assessment workload per filled vacancy does not rise.

Guardrail: proportion of listings receiving zero applications does not rise.

Notice. Completion rate was not a bad measure. It was a leading indicator being used as an outcome. That distinction is the whole of stage 3.

Nothing in this case required unusual insight to avoid. A single guardrail on a downstream outcome would have surfaced the problem in the first month instead of the sixth. That is the entire argument for defining the measure set before delivery rather than after.

The mandated identity step

Product work against a constraint that will not move

Legislation requires identity verification before a participant record can be created. Research shows it is where the most vulnerable applicants abandon. The constraint is real and is not going to change this financial year. This case is about what product management can still do when the obvious answer — remove the step — is not available.

Stage 2 — Establishing what is actually binding

The team read the instrument rather than the summary of it. It requires that identity be established to a specified assurance level before a record is created. It does not specify a document set, a channel, an order of steps, or that verification happen in one sitting. That distinction is the entire case. 'Identity must be verified' is binding. 'Applicants must upload three documents in a single session before proceeding' was an implementation decision made years earlier by people solving a different

[Skip to learning content](#) is though it were the law.

Binding: identity established to assurance level 2 before record creation
Not binding: which documents, which channel, what order, single session
Source: s.14(2), read directly – not the process guide summarising it

Notice. Constraints arrive bundled with the implementation someone chose for them. Separating the two is often the whole of the available design space, and it costs an afternoon of reading.

Stage 2 — Finding who the step actually excludes

Disaggregated, the rate was not general at all. Applicants with a current driver licence and a stable address abandoned at 6%. Applicants without one — people recently released from custody, people escaping family violence with no documents in their own name, people with no fixed address — abandoned at 74%. Observation with twelve people in the second group found the failure was not comprehension. They understood the page. They did not have the documents, could not obtain them quickly, and the service offered no path that acknowledged this.

Abandonment at identity step
with standard documents 6% (n=1,240)
without standard documents 74% (n=310)
The aggregate 31% describes nobody.

Notice. An average across two populations with different experiences describes neither. Disaggregation before intervention is the difference between fixing a page and finding the actual exclusion.

Stage 4 — Options when the constraint stays

The team listed options against the binding requirement rather than the inherited process. Verification still had to happen before a record existed — but nothing required the applicant to be alone, unaided, in one sitting, or online. Three options survived: an assisted pathway where a caseworker completes verification with the applicant present; a save-and-return state so a partial application persists while documents are obtained; and a referral to the existing documentation-support service, which already existed and which no one in the journey had ever been told about.

Option	Binding req met?	Cost	Reaches the 74%?
Rewrite the screen	yes	low	no
Assisted pathway	yes	medium	yes
Save and return	yes	medium	partly
	NO	--	--

[Skip to learning content](#)

Notice. The option that removes the constraint is usually the only one anybody names, and it is usually the one that is unavailable. Listing options against what is actually binding produces three you had not considered.

Stage 8 — Taking the evidence to the policy owner

The team presented the disaggregated abandonment data to the policy area as a finding, not a complaint, and did not ask for an exemption. The framing was: here is who your requirement is currently excluding, here is what we can do inside it, and here is what only you can change. The policy owner had never seen the requirement's effect on people without documents, because nothing in the assurance process reported it. They kept the assurance level and revised the guidance to name the assisted pathway as an acceptable route — which then applied across every service using the instrument.

Notice. A policy owner is not the opposition. They are usually the person with the least visibility of what their instrument does at the counter, because nothing routinely reports it back to them.

Stage 3 — Measuring the thing that mattered

The team measured completion for the group without standard documents specifically, and paired it with a guardrail on assisted-channel wait times — because a pathway that works only by pushing people into a queue that then takes six weeks has moved the problem rather than solved it. Abandonment in that group fell from 74% to 22% over two quarters. The overall rate moved from 31% to 14%, which is the number that would have been reported anyway and which would have looked like a modest usability win.

Outcome: applicants without standard documents complete identity verification
Baseline: 26% completion
Target: 60% by Q4
Actual: 78%
Guardrail: assisted-channel wait $\leq$ 5 working days (held at 3)

Notice. The headline would have improved on its own and told you almost nothing. The measure that made the work legible was the one scoped to the people it was for.

Nothing here required the constraint to move, and the constraint did not move. What moved was the assumption that the inherited process was the requirement. That distinction — binding versus

[Skip to learning content](#) every piece of government

product work, and it is where most of the real design space is hiding.

The service that quietly stopped working

What Live costs when nobody owns it

A claims service launched well, met every target, and the project closed. Eighteen months later completion had drifted down eight points with no release in that period. This case is about what a product manager does when the problem is that nobody has been looking — and about why the Beta-to-Live handover is where services are most often lost.

Stage 6 — The handover that recorded everything except ownership

The handover pack was thorough. It transferred the ability to keep the service running — incidents, availability, patching — to a team accountable for exactly that. What it did not transfer was accountability for whether the service still worked for the people using it. Nobody was named against completion rate. No cadence existed at which anyone would look. Operations were meeting every commitment they had been given, and the service was degrading, and both of those were true at once.

Transferred: incident response, availability, patching, support rota
Not transferred: completion rate, user research, the backlog, the outcome
Named owner for service performance: (blank)

Notice. Operational ownership and product ownership are different things, and a closure pack that transfers only the first will pass every review. The gap is invisible until a measure moves.

Stage 3 — Reading a drift with no release to blame

The code had not changed in the period. That rules out the design as a cause of the *change*, whatever its merits: a static system producing a declining result means something around it moved. The team worked through the four things that move underneath a stable service. Users: a new cohort had entered following an eligibility expansion. Policy: a rule change had added a condition. Upstream: a register had altered a field's format. Volume: a seasonal peak had pushed a queue past a threshold with no alarm on it. Three of the four had happened.

Completion, monthly (no releases in period)

Mar 82% #####

[Skip to learning content](#)

##

Dec 74% #####

Candidate causes: users [yes] policy [yes] upstream [yes] volume [no]

Notice. A drift with no release is diagnostic, not mysterious. It narrows the cause to the environment, and the environment has a short and checkable list.

Stage 2 — Which of the three actually caused it

The team segmented completion by cohort and by date. The eligibility expansion had brought in applicants whose circumstances the form handled poorly, but their completion was flat, not falling. The upstream field change affected a validation path used by 2% of applicants. The policy rule change matched the timing exactly, and the decline was concentrated among applicants the change was specifically intended to help — because the form asked about the new condition in language lifted directly from the instrument.

Segment	Mar	Dec	Delta
Pre-existing cohort	83%	76%	-7
New eligibility cohort	71%	70%	-1
Affected by upstream field	79%	77%	-2
Subject to new condition	80%	58%	-22 <-- here

Notice. Three plausible causes and one real one is the normal ratio. Segmenting before acting cost a week and prevented two changes that would have been risk without benefit.

Stage 7 — Fixing the thing that let it happen

The wording change took a day and recovered the completion rate within a month. The team also proposed the structural fix, which was harder to get funded because nothing was visibly broken any more. What they asked for was small: a named product owner for the service, four measures reported monthly, and a standing half-day each month to look at them. Not a team, not a project — an owner and a cadence.

Fix:	reword the condition question in plain language	(1 day)
Structural:	named owner for service performance	(0.1 FTE)
	four measures, reported monthly	
	half-day monthly review, standing	
Detection:	four months -> target one month	

Notice. The wording was the fault. The absence of an owner was the reason it took four months to find. Fixing only the first guarantees the

[Skip to learning content](#)

Stage 9 — Reporting it without burying it

The team led with the gap in detection rather than the fix. The sentence was: a policy change reduced completion for the group it was meant to help, it took us four months to notice because nobody owned the measure, we have fixed the cause in a day, and we are asking for the owner so the next one takes a month. That framing made the structural request fundable. Leading with the recovery would have made it a story about competence, and competence does not need investment.

Notice. How a result is framed determines what happens next. Leading with the fix closes the item; leading with the detection gap opens the decision that prevents the recurrence.

This service was never badly built and was never badly run. It was unowned, which is a different failure and a much more common one. The Beta-to-Live transition is where it happens, it happens quietly, and the cost is measured in how long it takes anyone to notice.

[Skip to learning content](#)

Templates

Product vision

For [users], create a future where [meaningful state], so that [public/user/organisational value].

Example. For employment-service providers, create a future where application progress is understandable and actionable, so they can support participants confidently with less avoidable effort.

Problem statement

Our [who] are currently experiencing [what] when trying to [goal]. This occurs in [where] and has been happening since [when]. This is critical because [why / impact]. Evidence: [sources].

Example. Our providers are currently experiencing an inability to determine application progress when trying to support participants. This occurs across the provider portal and the support line, and has been happening since the 2023 platform split. This is critical because it generates avoidable support demand and leaves participants without answers. Evidence: call-reason coding, 12 interviews, journey observation.

This follows the deck's who/what/goal/where/when/why structure from slide 30.

How might we

How might we [intended action] for [who] so that [desired outcome], while [important guardrail]?

Example. How might we make application progress understandable for providers so they can act without assistance, while preserving clear escalation for unresolved cases?

The deck's format (slide 31) omits the guardrail clause. It is added here because government services almost always have a group whose access must not be traded away.

[Skip to learning content](#)

Hypothesis

We believe [intervention] for [audience] will [outcome]. We will know this is true when we see [signal], and false if [disconfirming signal], without [guardrail failure].

Example. We believe plain-language status and next-step guidance will reduce avoidable calls for providers. We will know it is true if successful self-service rises above 65%, and false if call volume holds while page views rise — without unresolved-case escalation dropping below 95%.

The deck's version (slide 32) has no disconfirming signal and no guardrail. Both are added here: a hypothesis you cannot imagine being wrong is not testable.

Objective and key results

Objective: [meaningful direction]. KR1: move [metric] from [baseline] to [target] by [date]. KR2: [second measure]. Guardrail: [must not worsen]. Grade at review: [1-4] because [insight].

Example. Objective: Make application progress understandable without assistance. KR1: successful self-service 42% → 70% by June. KR2: avoidable status calls down 30%. Guardrail: unresolved-case escalation stays above 95%.

Experiment brief

Decision: [what will change]. Riskiest assumption: [claim]. Method: [test]. Participants/data: [sample]. Success/stop signals: [thresholds decided in advance].

Example. Decision: whether to invest in live status integration. Assumption: the proposed information resolves the main uncertainty. Test: realistic prototype with 8 representative providers using manually supplied data. Proceed if 6 of 8 correctly state the next action unprompted.

[Skip to learning content](#)

Epic

Epic hypothesis: [outcome we believe this delivers]. Description: [scope]. Business outcomes and benefits: [measurable outcomes tracked, linked to the business case]. Objective: [the objective it serves].

Example. Epic hypothesis: consolidating application status into a single provider view will materially reduce avoidable support demand. Benefits: 30% reduction in status-related contacts; linked to business-case benefit B3. Objective: reduce administrative effort for providers.

Epics span many PIs and multiple teams. Written to the deck's minimum detail, slide 61.

Feature

Feature hypothesis: [user outcome expected]. Sizing estimate: [relative size]. Acceptance criteria: [conditions]. In scope / out of scope: [boundaries]. Feature measurement: [metric, e.g. 30% reduction in processing time]. Risks, issues, dependencies: [items].

Example. Feature hypothesis: providers shown a plain-language status with a next action will resolve their query without contacting support. Measurement: successful self-service rate for status queries, target 65% in the first PI after release. Out of scope: payment status. Dependency: status API availability from the assessment platform.

Features are demonstrable within a PI. Feature measurement is the field most often skipped and the one that decides whether you learn anything.

User story with evidence

As a [persona], I want [capability] so that [reason]. Acceptance: [observable conditions]. Story points: [estimate]. Area path: [team/product]. Iteration path: [sprint]. Outcome link: [measure].

Example. As a provider, I want to see the current application status and the required next action so I can support the participant without calling. Acceptance: status meaning is available to screen readers, a timestamp is shown, and the escalation path is visible. Outcome link: successful self-service rate.

[Skip to learning content](#)

Evidence-based gate brief

Decision requested: [decision]. Phase gate: [Pre-Approval / Pre-Delivery / Delivery / Closure]. Outcome: [result sought]. Evidence: [what supports it]. Uncertainty: [what remains]. Risks and standards: [material items, including DSS criteria]. Ownership after this gate: [named owner]. Next test or investment: [proposal].

Example. Proceed to a controlled Beta after accessibility remediation and baseline capture. Evidence supports comprehension; data freshness remains a material operational risk. Enduring owner named for post-Live performance.

[Skip to learning content](#)

Field guide

DES delivery phases

The agreed end-to-end pipeline for all DES projects, with stage gates between phases. Distinct from the DTA service phases and from the Digital Service Standard.

Pre-Approval

Establishing the case for investment. Product contribution: problem definition, user evidence, outcome hypotheses, options and risks.

Pre-Delivery

Planning with the right amount of detail and establishing the delivery runway. Product contribution: refined backlog, validated approach, measures and baselines defined before build.

Delivery

Building and releasing increments. Product contribution: ordering the backlog, protecting the outcome, running discovery an increment ahead.

Closure

Transition and completion. Product contribution: handing over enduring ownership, measures and an improvement cadence — the phase where product risk is highest.

DTA service design and delivery phases

The government service process the product lifecycle aligns to. The deck presents Beta and Live combined; they are separated here because the ownership question at go-live is where services most often fail.

Discovery

Understand the service landscape, user needs, policy intent and technology constraints.

Alpha

Explore several approaches and test hypotheses through prototypes and research.

Beta

Build and trial an accessible, secure service based on successful Alpha learning.

Live

Operate the service and continue improving through performance data and user feedback.

[Skip to learning content](#)

The seven principles in practice

DES operating principles and what each one changes about day-to-day product work.

1. Start with the problem, not the solution

Understand user needs, policy intent and root causes before committing. Discovery comes before build.

2. Validate before you scale

Reduce risk through prototypes, testing and incremental releases.

3. Deliver value early and often

Prefer incremental delivery of valuable outcomes over large, infrequent releases.

4. Decisions belong closest to the problem

People with the greatest understanding of users and delivery realities make day-to-day decisions.

5. Focus on outcomes, not outputs

Outcomes over features, benefits over deliverables, value over activity.

6. Design for the whole service

Optimise end-to-end experience across channels and systems. Reuse before rebuild.

7. Learn continuously

Discovery, validation and improvement continue throughout the life of a service.

Ways of Working — behaviours

The behavioural half of DES ways of working. The other half is the Digital Delivery Framework.

We put user needs at the centre

Decisions are grounded in what users actually do and need.

We are accountable and take ownership

Issues are owned and solved, not routed.

We empower our people

Authority sits with the people doing the work.

We connect and work across boundaries

Whole-of-service beats local optimisation.

We do the basics brilliantly

Reliability and quality before novelty.

We are innovative and contemporary

Current practice, adopted deliberately rather than fashionably.

Can [Skip to learning content](#)

The DES planning rhythm and the minimum detail each backlog level must carry. Area path and iteration path are Azure DevOps fields.

Sprint

2 weeks / 10 working days. Delivers small increments of releasable value.

Program Increment (PI)

10 weeks / 50 working days. Delivers key business outcomes. Five sprints per PI.

PI flow

Business outcome → candidate features → refinement → increment planning → delivery → measurement.

Epic fields

Epic hypothesis; description; business outcomes and benefits; objective.

Feature fields

Hypothesis; sizing estimate; acceptance criteria; in/out of scope; feature measurement; risks, issues and dependencies.

Story fields

As a [persona] I want [goal] so that [reason]; story points; area path; iteration path.

Not Scrum terms

Epic, feature and Program Increment come from scaled frameworks and local practice. Scrum defines Product Goal, Product Backlog, Sprint Backlog and Increment.

Roles around the product manager

Who owns what across governance, coordination and delivery.

Senior Responsible Officer

Owns project outcomes, the delivery roadmap and championing the change.

Senior Supplier

Supplier-side commitment and capability.

Project Board

Investment and material risk decisions.

Lead Delivery Manager

Delivery across teams; removing systemic impediments.

Product Manager

Outcomes, priorities and product direction. Asks: are we working on the right things?

Product Owner

Building, curating and prioritising a refined, validated backlog. Asks: do teams have what they need?

UX [Skip to learning content](#)

User research, service and interaction design.

Solution Architect

Technical approach and integrity.

Scrum Master

Team process, flow and impediment removal.

BAs, developers, testers

Making working increments.

Prioritisation reference

WSJF components and the impact/effort quadrants, for use in refinement.

WSJF formula

Cost of delay ÷ job size. Higher sequences earlier.

Cost of delay

User and business value + time criticality + risk reduction or opportunity enablement.

Job size

Relative effort or duration. The denominator — which is why small valuable work moves early.

Quick wins

High impact, low effort. Do first.

Big-ticket items

High impact, high effort. Plan deliberately.

Fill-in jobs

Low impact, low effort. Do when capacity allows.

Thankless tasks

Low impact, high effort. Challenge whether they should be done at all.

Judgement still applies

Dependencies, safety, legislated mandates and confidence can and should override a score.

[Skip to learning content](#)

Glossary

Each entry names the body of practice the term comes from. Epics, features and Program Increments are not Scrum terms, and the DES delivery phases are not the DTA service phases.

Acceptance criteria

GENERAL

The observable conditions that must hold for a story or feature to be considered done. They answer 'is it built', not 'should we build it'.

Alpha

GOVERNMENT

The DTA phase for exploring and testing several approaches through prototypes and research, before committing to a Beta service.

Area path

SAFE

An Azure DevOps field assigning a work item to a team or product. Required on stories in the DES backlog.

Assumption

GENERAL

Something believed true but not yet supported strongly enough for the decision being made. The riskiest one is what you test first.

Beta

GOVERNMENT

The DTA phase for building and trialling an accessible, secure service with real users — the first test under real operating conditions.

Closure

DECK

The fourth DES delivery phase. Transition and completion — and where product risk is highest, because the project ends but the product does not.

Cost of delay

SAFE

In WSJF, user and business value plus time criticality plus risk reduction or opportunity enablement. The numerator, divided by job size.

Created need

DECK

Something users are forced to do because of policy or the way government works. A need manufactured by the system, not held by the user.

Decision rights

GENERAL

Explicit agreement about who decides, who contributes evidence, who is consulted, and what threshold requires escalation.

Desirability

GENERAL

Whether the problem matters and people will choose or benefit from the proposed response. The human lens in DVF.

Digital Service Standard

GOVERNMENT

The Australian Government's current 10-criterion standard, reduced from 13. Fully in effect: new services from 1 July 2024, pre-existing public-facing services from 1 July 2025.

Discovery

DECK

Work that reduces uncertainty about users, problems, opportunities and solution risks. Produces insight and hypotheses — never requirements or features.

Distributional harm

GENERAL

An intervention that improves the average while worsening outcomes for a specific group. Guardrails on disaggregated measures reveal it.

DVF

GENERAL

Desirable, Viable, Feasible — a three-lens filter applied before investing. when one perspective dominates.

[Skip to learning content](#)

Epic	SAFE	A measurable business outcome spanning many PIs and teams. Not a Scrum term. Requires a hypothesis, description, business outcomes and benefits, and an objective.
Feasibility	GENERAL	Whether the team can build and operate the solution with available technology, data, skills and time. The technology lens in DVF.
Feature	SAFE	A smaller tangible outcome contributing to an epic, demonstrable within a PI. Requires a hypothesis, sizing, acceptance criteria, scope boundaries, measurement, and risks.
Feature measurement	DECK	The metric that shows whether a delivered feature worked, e.g. '30% reduction in processing time'. The field most often skipped and the one that decides whether you learn anything.
Guardrail	GENERAL	A measure that detects unacceptable harm, trade-offs or displacement while pursuing the primary outcome. It must not worsen; it is not what you are improving.
How might we	DECK	A design-thinking reframe turning a problem into an actionable opportunity, without prescribing a solution.
Hypothesis	DECK	A testable statement linking a proposed action for a group to an expected outcome and observable evidence — including what would show it false.
Increment	SCRUM	A usable step toward the Product Goal. Each increment is additive to all prior increments and must be verified.
Iteration path	SAFE	An Azure DevOps field aligning a story to the sprint in which delivery is expected. Misaligned iteration paths make PI reporting silently drift.
Lagging indicator	DECK	A measure of past performance — customer satisfaction, outcome achieved. Confirms the result after it has happened.
Leading indicator	DECK	A predictive measure of future performance — feature usage, completion rate. Moves early, but must itself be validated.
Lean Canvas	GENERAL	A one-page model of problem, users, solution ideas, business outcomes, hypotheses and riskiest assumptions. Its function is to expose what to test first.
Live	GOVERNMENT	The DTA phase for operating the service and continuing to improve it through performance data and user feedback. Needs an owner, measures and a cadence.
MVP	DECK	The smallest coherent thing that delivers value and generates learning. Focused and intentional — not a poor-quality or unfinished product, and never an excuse to defer accessibility.
Objective	DECK	In OKRs, a qualitative, specific and motivating direction. Measured by two to five key results.

OKR [Skip to learning content](#) and Key Results. We will [objective], as measured by [key results, metrics, actions]. Graded 1 to 4 at review.

Opportunity GENERAL	A problem or need reframed as something the team could act on, before any solution has been chosen.
Outcome DECK	A meaningful change in behaviour, performance, experience, risk or value resulting from the work. The why.
Output DECK	The thing produced or delivered — a feature, form, policy or release. The what. Can be delivered perfectly and change nothing.
Persona DECK	A research-grounded synthesis of user goals, needs, behaviours, motivations, challenges and context. Not a demographic stereotype.
Pre-Approval DECK	The first DES delivery phase. Establishing the case for investment: problem definition, user evidence, outcome hypotheses, options and risks.
Pre-Delivery DECK	The second DES delivery phase. Planning with the right amount of detail and establishing the runway: refined backlog, validated approach, baselines defined before build.
Product DECK	An enduring vehicle for delivering value to identifiable users, with ongoing ownership and improvement. May be software, policy, operational procedures, or a combination.
Product Backlog SCRUM	An emergent, ordered list of what is needed to improve the product; the Scrum Team's single source of work. Anyone may add; product management orders.
Product Goal SCRUM	The long-term objective for the Scrum Team and the target against which the Product Backlog emerges.
Product Manager DECK	Holds vision, strategy, outcomes, roadmaps and stakeholder alignment. Asks: are we working on the right things?
Product Owner SCRUM	Holds backlog management, refinement, acceptance criteria and sprint support. Asks: do teams have what they need to deliver? Must not become a requirements courier.
Product trio GENERAL	Product manager (viable), designer (desirable) and technology lead (feasible) collaborating on discovery. A working pattern, not an approval committee.
Program Increment SAFE	Ten weeks / 50 working days in the DES cadence, delivering key business outcomes. Five two-week sprints fit inside one. Not a Scrum term.
Project DECK	A temporary structure for coordinating investment and delivery toward a defined change. Ends; the product does not.
Root cause analysis DECK	Working back from a visible symptom to the structural cause, commonly by asking why repeatedly. Prevents solving the first thing you noticed.
Senior Responsible Of	Owns the project outcomes, the delivery roadmap and championing the product decisions that alter the outcome need the SRO.

[Skip to learning content](#)

Service	DECK	The series of interactions that helps someone do something. A product is a tool created to deliver the service.
Service blueprint	DECK	A view connecting user actions and front-stage interactions to back-stage activities, supporting systems and processes.
Spike	GENERAL	A bounded piece of technical investigation run against representative interfaces and data, to answer a feasibility question before committing.
Sprint	SCRUM	Two weeks / 10 working days in the DES cadence, delivering small increments of releasable value.
Story	GENERAL	A small slice a team can complete and validate in a sprint. As a [persona] I want [goal] so that [reason], plus points, area path and iteration path.
Story points	GENERAL	A relative estimate of size and complexity, used for forecasting throughput. Not hours, and not a productivity measure.
Triangulation	GENERAL	Comparing evidence from different methods, because stated preference, observed behaviour and system constraints routinely disagree.
Vanity metric	GENERAL	A number that reliably rises and never changes a decision. If no value would make you act differently, it is not worth reporting.
Viability	GENERAL	Whether policy, operational, legal, financial and organisational conditions can sustain the solution. The business and policy lens in DVF.
Vision	DECK	A description of the future state the product or service is trying to create. The destination, not the route.
Ways of Working	DECK	The DES behaviours plus the Digital Delivery Framework — an end-to-end, repeatable delivery pipeline underpinned by seven principles.
Whole-of-service	DECK	Optimising the end-to-end experience across channels and systems, including assisted and non-digital, rather than one interface or project.
WSJF	SAFE	Weighted Shortest Job First: relative cost of delay divided by relative job duration or size. A comparative sequencing aid, not a guarantee.

[Skip to learning content](#)

Sources

All links freely readable — no paywalled papers, no books you have to buy.

1. **Product Management Fundamentals — 12AUG2026** — DEWR / Digital Experience and Solutions Division
Primary course spine, 98 slides across nine sections. Slide ranges are shown on each stage.
2. **Digital Service Standard, Version 2.0** — Australian Government / Digital Transformation Agency
Current standard: 10 criteria, reduced from the former 13. Fully in effect — new services from 1 July 2024, and public-facing services that existed before that date from 1 July 2025.
<https://www.digital.gov.au/policy/digital-experience/digital-service-standard>
3. **The Scrum Guide (2020)** — Ken Schwaber and Jeff Sutherland
Authoritative definitions for Product Goal, Product Backlog, Sprint Goal and Increment. Note that Scrum does not define epics, features or Program Increments.
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>
4. **Weighted Shortest Job First** — Scaled Agile Framework
Defines WSJF as relative cost of delay divided by relative job duration/size, and cost of delay as user/business value plus time criticality plus risk reduction or opportunity enablement.
<https://framework.scaledagile.com/wsjf/>
5. **Opportunity Solution Trees** — Teresa Torres, Product Talk
Free, full article. Mapping a clear outcome to opportunities, then to solutions and assumption tests, by a product trio.
<https://www.producttalk.org/2016/08/opportunity-solution-tree/>
6. **The Product Operating Model: An Introduction** — Marty Cagan, Silicon Valley Product Group
Outcome-accountable teams and the value, usability, feasibility and viability risks addressed in discovery.
<https://www.svpg.com/product-vs-feature-teams/>
7. **What, exactly, is Product Management?** — Martin Eriksson, Mind the Product
Origin of the business / technology / user-experience intersection
[Skip to learning content](#) of the source deck, where it appears
<https://www.mindtheproduct.com/what-exactly-is-a-product-manager/>

8. **Introduction to Design Thinking** — IDEO
Human-centred innovation balancing desirability, feasibility and viability.
<https://designthinking.ideo.com/introduction>
9. **What is a Lean Canvas?** — Ash Maurya
The canvas as a one-page model of assumptions to expose and test, adapted on slide 33 of the deck.
<https://ashmaurya.com/blog/what-is-lean-canvas>
10. **Product strategy and the build trap** — Melissa Perri
Free articles. Named in the deck's recommended reading (slide 97); the source of the 'build trap' framing behind outputs-versus-outcomes.
<https://melissaperri.com/blog>
11. **Introduction to Product Management (UK Government)** — Central Digital and Data Office with GDS, on FutureLearn
Free. The closest public comparator to this course: product management written for government delivery, built by CDDO with GDS and departmental colleagues. Useful for seeing which parts of the practice are common across jurisdictions and which are specific to DES.
<https://digitalpeople.blog.gov.uk/2024/10/15/introduction-to-product-management-now-available-on-futurelearn/>
12. **GOV.UK Service Manual** — Government Digital Service
Free. The reference standard for digital service delivery in government, and the model the Australian Digital Service Standard sits alongside. Read it for the phase guidance and the communities of practice.
<https://www.gov.uk/service-manual>
13. **Product Roadmaps and Product Strategy** — Roman Pichler
Named in the deck's recommended reading (slide 97). Goal-oriented roadmaps as an alternative to dated feature lists.
<https://www.romanpichler.com/blog/goal-oriented-agile-product-roadmap/>
14. **Test-Enhanced Learning** — Roediger and Karpicke (2006)
Retrieving knowledge strengthens later retention more effectively than passive restudy over longer delays.
<https://pubmed.ncbi.nlm.nih.gov/16507066/>
15. **Distributed Practice in Verbal Recall Tasks** — Cepeda et al. (2006)
Quantitative review supporting study distributed across time rather than massed into one sitting.
<https://pubmed.ncbi.nlm.nih.gov/16719566/>
16. **Improving Students' Learning With Effective Learning Techniques** — Dunlosky et al. (2013)
Dated practice testing and distributed practice as high-utility. Note that it makes you retrieve.
<https://pubmed.ncbi.nlm.nih.gov/26173288/>

[Skip to learning content](#)

Product Management Fundamentals — unofficial internal learning aid. Built from Product Management Fundamentals — 12AUG2026, by Simon Morris. Content reviewed 15 August 2026. Not an official Australian Government publication.

Internal learning aid · not an official Australian Government publication ·